

Node.js Complete Interview Master Book

10 Years Experience • 120+ Questions • Easy-to-Memorise • Senior-Level Answers

Interview formula: Definition → Internal working → Practical example → Trade-off → Production decision. Avoid giving only textbook definitions.

1. Node.js Fundamentals

1. What is Node.js?

Easy answer: Node.js is a JavaScript runtime built on V8 that lets JavaScript run outside the browser. Its major strength is non-blocking I/O and an event-driven architecture.

10-year insight: Do not say Node.js is simply 'single-threaded'. JavaScript runs mainly on one thread, while I/O is coordinated by the OS/libuv and selected work uses a thread pool.

Likely cross-question: Why is Node.js good for APIs but weaker for CPU-heavy work?

2. Why is Node.js single-threaded?

Easy answer: Node.js uses one main JavaScript execution thread and an Event Loop to coordinate asynchronous work.

10-year insight: This reduces thread-management overhead and makes concurrent I/O efficient, but long synchronous JavaScript blocks all requests handled by that process.

Likely cross-question: How can Node.js handle thousands of concurrent requests with one JS thread?

3. How does Node.js handle multiple requests?

Easy answer: Requests are accepted by the server, asynchronous I/O is delegated, and the Event Loop executes callbacks when operations complete.

10-year insight: The scalability comes from not waiting synchronously for I/O. Concurrency is not the same as parallel JavaScript execution.

Likely cross-question: What happens if one request performs a 5-second CPU calculation?

4. What is V8?

Easy answer: V8 is Google's JavaScript engine used by Node.js to parse, compile and execute JavaScript and manage garbage collection.

10-year insight: V8 executes JavaScript; it is not the component that provides Node's complete asynchronous I/O model.

Likely cross-question: What is libuv then?

5. What is libuv?

Easy answer: libuv is a cross-platform library providing the Event Loop, asynchronous I/O abstractions and a thread pool used by some Node.js operations.

10-year insight: Remember: V8 = JavaScript engine; libuv = async infrastructure.

Likely cross-question: Which Node.js operations use the thread pool?

6. Advantages of Node.js?

Easy answer: Fast for I/O-heavy workloads, large ecosystem, JavaScript/TypeScript across frontend and backend, streaming support, and good concurrency.

10-year insight: The important advantage is efficient I/O concurrency, not simply raw CPU speed.

Likely cross-question: What are the disadvantages?

7. Disadvantages of Node.js?

Easy answer: CPU-heavy synchronous work can block the Event Loop; dependency quality varies; large systems need careful async error handling and observability.

10-year insight: Senior engineers explain when not to use a technology.

Likely cross-question: When would you choose Java/Spring instead?

8. When should you not use Node.js?

Easy answer: A workload dominated by CPU-heavy JavaScript is a poor fit unless computation is isolated using Worker Threads or another service.

10-year insight: The decision should be based on workload characteristics rather than technology preference.

Likely cross-question: How would you handle CPU-heavy work in Node?

9. What happens when node app.js runs?

Easy answer: Node starts the process, initializes V8, loads the entry module, sets up required runtime structures, executes top-level JavaScript, registers async work, and enters/continues the Event Loop while work remains.

10-year insight: The useful interview point is understanding that top-level code executes before asynchronous callbacks.

Likely cross-question: Where does the Event Loop fit into this lifecycle?

10. Synchronous vs asynchronous APIs?

Easy answer: Synchronous APIs block the current JavaScript execution until they finish; asynchronous APIs allow other work to proceed while the operation completes.

10-year insight: Avoid synchronous filesystem/network operations on request paths unless there is a deliberate reason.

Likely cross-question: Why can sync APIs be dangerous in an API server?

2. Event Loop

11. Explain the Node.js Event Loop.

Easy answer: The Event Loop repeatedly checks phases/queues and executes ready callbacks, allowing Node.js to continue processing while asynchronous operations are pending.

10-year insight: Think of it as a traffic controller, not a worker doing every I/O operation itself.

Likely cross-question: What are the Event Loop phases?

12. What are the Event Loop phases?

Easy answer: Common phases include timers, pending callbacks, idle/prepare, poll, check, and close callbacks.

10-year insight: For interviews, focus on what each phase means rather than memorizing implementation trivia.

Likely cross-question: Where does setImmediate run?

13. process.nextTick vs Promise.then?

Easy answer: Both can schedule microtask-like follow-up work, but nextTick uses Node's special nextTick queue and has higher priority than the normal microtask queue.

10-year insight: Excessive nextTick scheduling can starve I/O and other phases.

Likely cross-question: Which runs first and why?

14. nextTick vs setImmediate?

Easy answer: nextTick runs before the Event Loop proceeds to normal phases; setImmediate runs in the check phase.

10-year insight: Use nextTick only when you specifically need Node's immediate post-operation semantics; do not recursively schedule it.

Likely cross-question: How can nextTick starve the Event Loop?

15. setTimeout vs setImmediate?

Easy answer: setTimeout schedules a timer after its threshold; setImmediate schedules a callback in the check phase.

10-year insight: The observed order depends on where they are scheduled and the current phase.

Likely cross-question: Why can their order change?

16. What are microtasks and macrotasks?

Easy answer: Microtasks are high-priority continuation work such as Promise reactions; Node also has nextTick. Timers, I/O callbacks and setImmediate are scheduled through Event Loop phases.

10-year insight: Microtasks can run before the Event Loop moves forward, so too many can delay I/O.

Likely cross-question: Why is an infinite Promise chain dangerous?

17. What is Event Loop starvation?

Easy answer: It occurs when the Event Loop cannot reach other work because higher-priority queues or long synchronous tasks keep consuming execution time.

10-year insight: Typical causes are CPU-heavy code, excessive `nextTick` or unbounded microtask chains.

Likely cross-question: How would you detect Event Loop lag?

18. What happens with CPU-intensive code?

Easy answer: The JavaScript thread stays busy, so other callbacks cannot execute and request latency increases.

10-year insight: Move CPU-heavy work to Worker Threads, another process, or another service.

Likely cross-question: Would adding more `async/await` solve it?

19. How do you prevent blocking?

Easy answer: Use async APIs, streaming, chunking, pagination, bounded work and Worker Threads for CPU-heavy computation.

10-year insight: Measure Event Loop delay so you can prove whether blocking is the problem.

Likely cross-question: How would you process a huge CSV safely?

20. Explain Event Loop with a practical example.

Easy answer: If an API starts a database call, Node does not sit idle waiting. It can process other requests; once the database result is ready, the continuation is queued and executed.

10-year insight: This is why Node handles I/O concurrency efficiently despite one main JS thread.

Likely cross-question: What changes when the DB driver itself blocks?

3. Async, Promises & Concurrency

21. Callbacks vs Promises vs `async/await`?

Easy answer: Callbacks pass a completion function; Promises represent eventual completion; `async/await` gives cleaner syntax for Promise-based flows.

10-year insight: `async/await` does not make code synchronous; it improves readability over asynchronous control flow.

Likely cross-question: How do you handle errors with `async/await`?

22. What happens internally with `async/await`?

Easy answer: An async function returns a Promise. `await` pauses that function's continuation until the awaited Promise settles; it does not block the JavaScript thread.

10-year insight: Other Event Loop work can continue while the Promise is pending.

Likely cross-question: Does `await` block Node.js?

23. How do you handle async errors?

Easy answer: Use `try/catch` around awaited operations, propagate meaningful errors, and centralize HTTP error handling in frameworks such as Express.

10-year insight: Preserve the original cause/context when wrapping errors.

Likely cross-question: How do you handle rejected Promises globally?

24. `Promise.all` vs `allSettled`?

Easy answer: `all` rejects as soon as one Promise rejects; `allSettled` waits for all and returns each status/result.

10-year insight: Choose based on business semantics: `all` when every result is required; `allSettled` for partial-success workflows.

Likely cross-question: Give a real-world use case for each.

25. Promise.race vs any?

Easy answer: race settles when the first Promise settles; any fulfills when the first Promise fulfills and rejects only when all reject.

10-year insight: race is useful for timeouts/competition; any is useful when any healthy source can satisfy the request.

Likely cross-question: How would you implement a timeout?

26. How do you execute 100 APIs safely?

Easy answer: Use bounded concurrency rather than firing all requests simultaneously. A queue/limiter can maintain a fixed number of active requests.

10-year insight: Concurrency limits protect your server and downstream services.

Likely cross-question: What number would you choose?

27. Sequential vs parallel execution?

Easy answer: Sequential waits for each operation before starting the next; parallel starts independent operations together.

10-year insight: Parallelize only independent work and respect downstream capacity.

Likely cross-question: When would sequential be necessary?

28. How do you implement concurrency control?

Easy answer: Maintain a queue and an active counter; start new work only when active work is below the configured limit.

10-year insight: In production, consider cancellation, retries, timeouts and backpressure as well.

Likely cross-question: How would you prevent a queue from growing forever?

4. Streams & Buffers

29. What are Streams?

Easy answer: Streams process data incrementally instead of loading the whole payload into memory.

10-year insight: They are especially valuable for large files, HTTP bodies, compression and transformations.

Likely cross-question: What are the four stream types?

30. Types of Streams?

Easy answer: Readable, Writable, Duplex and Transform.

10-year insight: Transform streams are especially useful for pipelines where input is transformed into output.

Likely cross-question: Give a Transform stream example.

31. What is a Buffer?

Easy answer: A Buffer represents raw bytes in Node.js and is used for binary data such as files and network payloads.

10-year insight: Large binary data should not be converted unnecessarily into strings.

Likely cross-question: Buffer vs string?

32. Why Streams for large files?

Easy answer: Reading a 5 GB file into memory can create severe memory pressure; streaming processes small chunks and keeps memory bounded.

10-year insight: Streaming changes the memory profile of the operation.

Likely cross-question: How would you send a 5 GB download?

33. What is backpressure?

Easy answer: Backpressure occurs when the consumer is slower than the producer. The pipeline must slow production instead of accumulating unlimited data.

10-year insight: It is one of the key reasons streams can safely handle large data.

Likely cross-question: How does pipe help?

34. How does pipe handle backpressure?

Easy answer: pipe connects streams and coordinates flow so the readable does not continuously overwhelm the writable.

10-year insight: Always respect stream lifecycle and errors in production pipelines.

Likely cross-question: What happens if the writable becomes slow?

35. How would you process a 5 GB file?

Easy answer: Use a Readable stream, process chunks/lines incrementally, and write results through a stream rather than loading the entire file.

10-year insight: Add error handling, backpressure and controlled concurrency if external calls are involved.

Likely cross-question: How would you process each line with an async API?

36. Stream error handling?

Easy answer: Listen for stream errors or use pipeline-based APIs that propagate failures through the pipeline.

10-year insight: A pipeline should have a clear success/failure lifecycle and cleanup strategy.

Likely cross-question: Why is pipeline safer than manual pipe chains?

5. Modules

37. CommonJS vs ES Modules?

Easy answer: CommonJS uses require/module.exports; ES Modules use import/export.

10-year insight: Understand interoperability and package configuration when working in mixed ecosystems.

Likely cross-question: How does Node decide module type?

38. require vs import?

Easy answer: require is CommonJS loading syntax; import is ESM syntax and supports static module analysis.

10-year insight: Do not treat them as simple syntax replacements; loading semantics differ.

Likely cross-question: Can you use both in one project?

39. What happens when a module is required?

Easy answer: Node resolves the module, loads and executes it if needed, and caches the result so subsequent requires reuse the cached module.

10-year insight: This explains why module-level singleton state can persist.

Likely cross-question: How do you clear module cache?

40. How does module caching work?

Easy answer: Once loaded, a module's exported value is cached by the runtime for subsequent imports/requires according to the module system.

10-year insight: Do not accidentally depend on mutable global module state in scalable designs.

Likely cross-question: Can module caching cause bugs?

41. module.exports vs exports?

Easy answer: exports initially references module.exports. Reassigning exports alone does not replace the module's exported object; assigning module.exports does.

10-year insight: Remember: module.exports is the actual export target.

Likely cross-question: What happens with exports = something?

42. How do you create a reusable module?

Easy answer: Expose a small, stable API and hide internal implementation details. Keep dependencies explicit and avoid mutable global state.

10-year insight: Good modules have one clear responsibility and are easy to test.

Likely cross-question: How would you structure a shared utility package?

6. Error Handling

43. How do you handle errors in Node?

Easy answer: Validate inputs, catch expected async failures, propagate errors with context, centralize HTTP error formatting, log safely, and monitor unexpected process-level failures.

10-year insight: Errors should be classified and handled according to recoverability.

Likely cross-question: Operational vs programmer error?

44. Operational vs programmer errors?

Easy answer: Operational errors are expected runtime failures such as timeouts or unavailable dependencies. Programmer errors are bugs such as invalid assumptions or undefined access.

10-year insight: Operational failures can often be handled; programmer errors usually require fixing and controlled restart.

Likely cross-question: Should every error be caught?

45. try/catch with async/await?

Easy answer: Awaited Promise rejections can be caught by try/catch surrounding the await.

10-year insight: Do not wrap every line blindly; place boundaries where you can actually handle or add context.

Likely cross-question: How do you centralize Express errors?

46. uncaughtException?

Easy answer: It means an exception escaped normal handling and reached the process-level handler.

10-year insight: Do not assume the process is safe after an unknown invariant-breaking error; use logging and controlled process restart.

Likely cross-question: Should the process continue?

47. unhandledRejection?

Easy answer: It means a Promise rejection has no rejection handler within the relevant turn/handling window.

10-year insight: Treat unexpected unhandled rejections as serious defects and ensure async code has deliberate error paths.

Likely cross-question: How would you monitor them?

48. Should you continue after uncaughtException?

Easy answer: Generally, do not rely on continuing a potentially corrupted process. Log context, stop accepting new work where appropriate, and let a supervisor restart it.

10-year insight: Graceful shutdown and process supervision are part of production reliability.

Likely cross-question: How do you perform graceful shutdown?

7. Express.js

49. What is Express middleware?

Easy answer: A function in the request pipeline that can inspect/modify request/response, end the response, or call next().

10-year insight: Middleware is ideal for cross-cutting concerns such as auth, validation, logging and correlation IDs.

Likely cross-question: Application vs router middleware?

50. How does middleware execution work?

Easy answer: Express executes middleware in registration order. Each middleware either ends the response or passes control to the next layer.

10-year insight: Order matters for authentication, parsing, validation and error handling.

Likely cross-question: What happens if next is never called?

51. Application vs router middleware?

Easy answer: Application middleware is registered broadly on the app; router middleware is scoped to a router or route group.

10-year insight: Scope middleware to reduce unintended behavior.

Likely cross-question: Where would auth middleware live?

52. Error-handling middleware?

Easy answer: Express error middleware uses the error-handling signature and centralizes conversion of failures into consistent responses.

10-year insight: Do not leak stack traces or sensitive details to clients in production.

Likely cross-question: How do async errors reach it?

53. Authentication vs authorization?

Easy answer: Authentication answers 'who are you?'; authorization answers 'what are you allowed to do?'

10-year insight: Keep them conceptually separate because a valid identity does not imply permission.

Likely cross-question: Give an RBAC example.

54. Structure a large Express app?

Easy answer: Use route/controller/service/repository layers, validation, middleware, configuration and shared utilities.

10-year insight: Keep business logic out of controllers and data access out of HTTP concerns.

Likely cross-question: Why separate controller and service?

55. Validate API requests?

Easy answer: Validate body, query and path parameters against an explicit schema before business logic runs.

10-year insight: Validation is both correctness and security.

Likely cross-question: Where should validation happen?

56. Implement rate limiting?

Easy answer: Track requests per identity/IP/key using a bounded, usually distributed mechanism and reject requests beyond policy.

10-year insight: In multi-instance deployments, shared state may be required.

Likely cross-question: Why can in-memory rate limiting fail in production?

57. Secure Express API?

Easy answer: Use TLS, validation, authentication/authorization, secure headers, rate limiting, safe database queries, secret management and dependency hygiene.

10-year insight: Security is layered; no single package makes an API secure.

Likely cross-question: What would you check next?

8. Performance

58. How identify a bottleneck?

Easy answer: Measure latency, CPU, memory, Event Loop lag, DB timings, downstream timings and throughput using metrics/traces/profiling.

10-year insight: Start with evidence rather than assumptions.

Likely cross-question: What metric tells you Event Loop health?

59. How find memory leaks?

Easy answer: Look for sustained heap growth after garbage collection, compare heap snapshots and identify retained objects/references.

10-year insight: Typical causes include unbounded caches, listeners, timers, globals and closures.

Likely cross-question: How do closures create leaks?

60. What causes high CPU?

Easy answer: CPU-heavy loops, expensive serialization, parsing, encryption/compression, regex or accidental infinite work can cause it.

10-year insight: Profile before optimizing; then isolate CPU work if necessary.

Likely cross-question: How would you fix CPU saturation?

61. What causes high memory?

Easy answer: Large objects, buffering huge payloads, caches, retained references, concurrency spikes or memory leaks.

10-year insight: High memory is not automatically a leak; inspect heap behavior over time.

Likely cross-question: How would you distinguish cache growth from a leak?

62. Optimize a slow API?

Easy answer: Break down latency by application, DB and downstream calls; remove unnecessary work, optimize queries, parallelize independent calls, cache carefully and reduce payload size.

10-year insight: Optimize the actual bottleneck, not the code that merely looks expensive.

Likely cross-question: How would tracing help?

63. Handle 10,000 concurrent requests?

Easy answer: Keep request work non-blocking, use efficient connection pools, protect dependencies with limits/timeouts, scale instances and use load balancing.

10-year insight: Concurrency must be bounded where downstream systems have finite capacity.

Likely cross-question: What becomes the bottleneck first?

64. What is clustering?

Easy answer: Cluster runs multiple Node.js processes so a service can use multiple CPU cores and gain process isolation.

10-year insight: Today, container/orchestrator-based horizontal scaling is often preferred, but the process model remains important.

Likely cross-question: Cluster vs Kubernetes replicas?

65. Cluster vs Worker Threads?

Easy answer: Cluster = multiple processes; Worker Threads = multiple JS execution threads within a process.

10-year insight: Use cluster/processes for isolation and scaling; workers for CPU-heavy parallel work.

Likely cross-question: Can workers share memory?

66. How scale horizontally?

Easy answer: Run multiple stateless instances behind a load balancer, externalize shared state, scale databases/cache appropriately, and add observability.

10-year insight: Scaling the application without scaling dependencies can simply move the bottleneck downstream.

Likely cross-question: What would you check next?

9. Cluster & Worker Threads

67. Why Worker Threads?

Easy answer: They allow CPU-intensive JavaScript to run away from the main Event Loop thread, improving responsiveness of the main process.

10-year insight: They are not a replacement for asynchronous I/O.

Likely cross-question: Give a CPU-heavy example.

68. What problem do Workers solve?

Easy answer: They solve CPU contention where a long computation would otherwise block the main JavaScript thread.

10-year insight: Use a worker pool for repeated jobs instead of spawning unlimited workers.

Likely cross-question: Why not create one worker per request?

69. Cluster vs Worker Threads?

Easy answer: Cluster gives process isolation and multi-process scaling; workers provide thread-level parallel computation.

10-year insight: Processes isolate memory failures better; workers are useful when computation needs parallelism within an application.

Likely cross-question: Which is better for image processing?

70. Multiple CPU cores?

Easy answer: One Node.js process does not automatically execute JavaScript simultaneously across all cores. Multiple processes or Worker Threads are needed for parallel CPU execution.

10-year insight: Modern deployment platforms often achieve this through replicas.

Likely cross-question: How does a load balancer distribute traffic?

71. What if a cluster worker crashes?

Easy answer: A supervising parent/orchestrator can detect failure and create a replacement worker/process.

10-year insight: Production systems should have health checks and restart policies.

Likely cross-question: How do you prevent crash loops?

72. CPU-intensive solution?

Easy answer: Move the computation to a Worker Thread pool or a dedicated worker/service, keeping request handlers responsive.

10-year insight: Add queueing, concurrency limits, timeouts and monitoring around the job system.

Likely cross-question: What would you check next?

10. Memory Management

73. How does Node memory work?

Easy answer: JavaScript objects live primarily in V8-managed heap memory, while Buffers and native resources can use memory outside the ordinary JS heap.

10-year insight: Monitor both heap and process-level memory.

Likely cross-question: What is RSS?

74. What is Garbage Collection?

Easy answer: Garbage collection identifies objects that are no longer reachable and reclaims their memory.

10-year insight: GC itself consumes CPU, so excessive allocation can create latency even without a leak.

Likely cross-question: Can GC prevent all memory leaks?

75. What is V8 heap?

Easy answer: It is the memory area managed by V8 for JavaScript objects and related runtime structures.

10-year insight: Heap limits are configurable and differ from total process memory.

Likely cross-question: How do you inspect heap usage?

76. Common memory leaks?

Easy answer: Global references, unbounded caches, retained closures, event listeners, timers, queues and objects referenced by long-lived structures.

10-year insight: Find the retaining path, not just the large object.

Likely cross-question: Why can an event listener leak memory?

77. Diagnose production leak?

Easy answer: Monitor memory over time, inspect GC behavior, capture heap snapshots, compare retained objects and correlate growth with traffic/code changes.

10-year insight: Take care with diagnostics in production because snapshots can be expensive.

Likely cross-question: Which tools can help?

78. Closures and memory?

Easy answer: A closure retains references needed by its function. If the closure itself remains reachable through a long-lived timer/listener/cache, those referenced objects can remain reachable too.

10-year insight: Closure is not inherently a leak; retention lifetime is the issue.

Likely cross-question: What would you check next?

11. Security

79. Secure Node API?

Easy answer: Use TLS, strong authentication, authorization, validation, safe DB access, rate limits, secure headers, secret management and dependency updates.

10-year insight: Security is defense in depth.

Likely cross-question: What is the most common API security mistake?

80. JWT vs sessions?

Easy answer: JWTs carry claims in a signed token and can support stateless APIs; sessions keep server-side session state and use a session identifier.

10-year insight: JWT does not automatically mean more secure; token lifetime, storage, revocation and architecture matter.

Likely cross-question: When would you choose sessions?

81. Prevent SQL injection?

Easy answer: Use parameterized queries/prepared statements and never concatenate untrusted input into SQL.

10-year insight: Validation is helpful but must not replace parameterization.

Likely cross-question: What about NoSQL injection?

82. Prevent NoSQL injection?

Easy answer: Use safe query construction, validate types/shapes and do not blindly pass client-controlled objects into database filters.

10-year insight: Treat request objects as untrusted data.

Likely cross-question: How would you validate a Mongo filter?

83. Prevent XSS?

Easy answer: Contextually encode output, sanitize untrusted HTML where HTML is actually required, and use secure browser policies.

10-year insight: API-only services still need to consider reflected/stored data consumed by web clients.

Likely cross-question: What is CSP?

84. Prevent CSRF?

Easy answer: For cookie-based authentication, use appropriate SameSite settings and CSRF tokens/Origin checks according to architecture.

10-year insight: CSRF is primarily a browser credential-attachment problem.

Likely cross-question: Does JWT always eliminate CSRF?

85. CORS?

Easy answer: CORS is a browser policy controlling which origins can access resources. Configure allowed origins, methods and credentials explicitly.

10-year insight: CORS is not an authentication mechanism.

Likely cross-question: Why is Access-Control-Allow-Origin '*' incompatible with credentialed requests?

86. Password storage?

Easy answer: Never store plaintext passwords. Use a dedicated password hashing algorithm such as Argon2id or bcrypt with appropriate parameters and unique salts.

10-year insight: Encryption and password hashing solve different problems.

Likely cross-question: Why not SHA-256 alone?

12. Database & API

87. Connect Node to databases?

Easy answer: Use a well-supported driver/ORM/query builder, configure connection pooling, handle startup/readiness and failures, and keep data access behind a clear boundary.

10-year insight: Do not create a new DB connection per request.

Likely cross-question: Why is pooling important?

88. Connection pooling?

Easy answer: A pool maintains reusable database connections so requests do not pay connection setup cost repeatedly.

10-year insight: Pool size should match DB capacity and application concurrency; larger is not automatically better.

Likely cross-question: What happens if the pool is exhausted?

89. DB connection failure?

Easy answer: Use timeouts, retries only where safe, health/readiness checks, circuit breaking where appropriate, clear errors and observability.

10-year insight: Do not hammer an unavailable database with unlimited retries.

Likely cross-question: How would you handle startup if DB is unavailable?

90. Optimize slow query?

Easy answer: Inspect the query plan, indexes, data volume, joins/filters, returned columns and pagination strategy.

10-year insight: Measure with production-like data; an index that helps one query can hurt writes.

Likely cross-question: What is an N+1 problem?

91. Pagination?

Easy answer: Offset pagination is simple but can become slower/inconsistent for changing large datasets. Cursor pagination uses a stable ordering key and scales better for many large datasets.

10-year insight: Choose based on UX and data access patterns.

Likely cross-question: Why is cursor pagination better for feeds?

92. Transactions?

Easy answer: A transaction groups related operations so they satisfy the database's transaction guarantees and can commit or roll back together.

10-year insight: Keep transactions short and avoid unnecessary network calls inside them.

Likely cross-question: How would you handle a transaction across microservices?

93. Design scalable REST API?

Easy answer: Define resource-oriented endpoints, validation, authentication, consistent errors, pagination, idempotency where needed, observability and versioning strategy.

10-year insight: API design includes failure behavior, not just endpoint naming.

Likely cross-question: How would you make POST idempotent?

94. API idempotency?

Easy answer: An idempotent operation produces the same intended result when repeated. For non-idempotent actions like payments/orders, use an idempotency key and durable request state.

10-year insight: Idempotency protects against client retries and network ambiguity.

Likely cross-question: Where should the idempotency key be stored?

13. Testing

95. Unit vs integration vs E2E?

Easy answer: Unit tests isolate small units; integration tests verify component interactions such as API+DB; E2E tests validate user/business flows across the system.

10-year insight: Use a balanced test pyramid rather than only expensive E2E tests.

Likely cross-question: What should be mocked?

96. Jest vs Mocha?

Easy answer: Both support JavaScript testing. Jest provides an integrated runner, assertions, mocks and utilities; Mocha is more modular and often paired with separate assertion/mock libraries.

10-year insight: The choice is ecosystem/team fit, not simply popularity.

Likely cross-question: How do you test async code?

97. Mock dependencies?

Easy answer: Mock external systems when testing unit behavior, but use integration tests against realistic boundaries to catch wiring/query issues.

10-year insight: Over-mocking can create tests that pass while production integration fails.

Likely cross-question: What should not be mocked?

98. Test async code?

Easy answer: Await the Promise or return it from the test and explicitly assert rejection/error cases.

10-year insight: Test timeout, retry and cancellation paths too—not just success.

Likely cross-question: How do you test a rejected Promise?

99. Test Express API?

Easy answer: Use unit tests for handlers/services and integration tests that send requests through the app with controlled dependencies/data.

10-year insight: Test authentication, validation, error responses and boundary behavior.

Likely cross-question: What would you check next?

14. Senior Production Scenarios

100. API suddenly goes from 200ms to 10s. What do you do?

Easy answer: Check recent deployments, traffic, latency percentiles, traces, Event Loop lag, CPU/memory, DB latency and downstream calls. Identify the largest latency contributor before changing code.

10-year insight: Say: “I would localize the bottleneck first rather than blindly scaling.”

Likely cross-question: What if only p99 increased?

101. CPU 100% without traffic increase?

Easy answer: Check profiles, recent code changes, infinite loops, serialization, regex, compression/encryption, GC pressure and worker/process behavior.

10-year insight: CPU saturation with stable traffic often points to changed work-per-request or pathological behavior.

Likely cross-question: How would you profile Node?

102. Memory rises every few hours?

Easy answer: Confirm sustained post-GC growth, capture heap snapshots, compare retained objects and identify the retaining reference.

10-year insight: Differentiate leak from legitimate cache/queue growth.

Likely cross-question: How would you prove the fix?

103. 50,000 requests/minute?

Easy answer: Keep handlers non-blocking, scale horizontally, use load balancing, tune DB pools, cache carefully, protect downstreams and add rate/concurrency limits.

10-year insight: Capacity planning must include dependencies.

Likely cross-question: How would you load test it?

104. API calls 5 services?

Easy answer: Run independent calls in parallel with bounded concurrency, use timeouts, propagate correlation IDs and define partial-failure behavior.

10-year insight: Parallelism reduces latency, but uncontrolled fan-out can overload dependencies.

Likely cross-question: What if one service is slow?

105. Downstream service unavailable?

Easy answer: Use timeout, bounded retry with exponential backoff where safe, circuit breaking/fallback where appropriate, and fail fast rather than consuming all local resources.

10-year insight: Reliability is about preventing cascading failure.

Likely cross-question: What is a circuit breaker?

106. Implement retry safely?

Easy answer: Retry only transient failures, use exponential backoff with jitter, cap attempts and total time, and ensure the operation is idempotent or protected by an idempotency mechanism.

10-year insight: Retries without limits amplify outages.

Likely cross-question: When should you never retry?

107. Prevent one request blocking others?

Easy answer: Avoid synchronous CPU work, enforce request timeouts, stream large data, limit concurrency and isolate CPU jobs.

10-year insight: Node's main risk is not request count; it is shared Event Loop contention.

Likely cross-question: How do you measure blocking?

108. Process millions of records?

Easy answer: Stream/chunk data, paginate database reads, batch writes, use bounded concurrency and checkpoints so a failure does not restart everything.

10-year insight: Backpressure and memory bounds are essential.

Likely cross-question: How would you make it resumable?

109. High availability Node service?

Easy answer: Run multiple instances across failure domains, load balance, use health checks, graceful shutdown, stateless design, resilient dependencies and observability.

10-year insight: HA is a system property, not a Node.js feature.

Likely cross-question: What does graceful shutdown do?

15. Coding Questions — What Interviewers Expect

110. Implement debounce.

Easy answer: Return a function that clears the previous timer and schedules a new invocation after the wait period.

10-year insight: Explain closure, timer lifecycle and cancellation. A senior answer also discusses leading/trailing options.

Likely cross-question: Where would debounce be used?

111. Implement throttle.

Easy answer: Allow execution at most once per configured interval, optionally scheduling a trailing call.

10-year insight: Discuss timestamp vs timer approaches and edge cases.

Likely cross-question: Throttle vs debounce?

112. Implement Promise.all.

Easy answer: Accept iterable inputs, preserve result order, resolve when all fulfill and reject when any rejects.

10-year insight: Discuss thenables, empty input and early rejection semantics.

Likely cross-question: What happens to already-started Promises after rejection?

113. Retry with exponential backoff.

Easy answer: Retry transient failures with increasing delays, a maximum attempt count and jitter.

10-year insight: Senior answer includes idempotency, timeout budget and retry amplification.

Likely cross-question: Why add jitter?

114. Concurrency limiter.

Easy answer: Maintain a queue and active counter; start new tasks only when active count is below the limit.

10-year insight: Discuss queue size, cancellation and fairness.

Likely cross-question: How would you prevent unbounded memory?

115. Implement EventEmitter.

Easy answer: Maintain event-to-listener collections, provide on/once/off/emit and invoke listeners when an event is emitted.

10-year insight: Discuss listener cleanup, synchronous emission and error events.

Likely cross-question: How can EventEmitter cause memory leaks?

116. LRU cache.

Easy answer: Use a Map to maintain key/value entries and update recency on access; evict the least recently used entry when capacity is exceeded.

10-year insight: Discuss TTL, memory limits and concurrent access.

Likely cross-question: Why is Map useful?

117. Rate limiter.

Easy answer: Track request counts/tokens per identity using a fixed/sliding window or token bucket and reject beyond the configured limit.

10-year insight: Distributed deployments need shared/distributed state or an external limiter.

Likely cross-question: Token bucket vs fixed window?

118. Flatten nested object.

Easy answer: Traverse nested values recursively or iteratively, build flattened keys and handle arrays/cycles according to requirements.

10-year insight: Ask about depth, arrays, circular references and prototype pollution before coding.

Likely cross-question: How do you avoid maximum call stack errors?

119. Process huge file.

Easy answer: Use streams and process chunks/lines incrementally rather than reading the entire file.

10-year insight: Discuss backpressure, errors and output flow.

Likely cross-question: How would you parallelize safely?

120. Duplicate records.

Easy answer: Use a Set/Map for O(n) average lookup when memory allows, or sort/stream/group when data exceeds memory.

10-year insight: Explain the time-memory trade-off.

Likely cross-question: What if the dataset is 100 GB?

121. REST API coding.

Easy answer: Separate route, controller, service and data layers; validate input, return consistent errors and use async error handling.

10-year insight: Senior candidates discuss idempotency, security and observability.

Likely cross-question: How would you test it?

122. Middleware coding.

Easy answer: Implement a function receiving request, response and next; perform the cross-cutting concern and call next or end the response.

10-year insight: Discuss ordering and error propagation.

Likely cross-question: What if middleware is async?

123. Promise timeout wrapper.

Easy answer: Race the operation against a timer Promise, reject when the timeout wins, and ideally propagate cancellation to the underlying operation.

10-year insight: Timeout alone does not necessarily cancel the underlying work.

Likely cross-question: How would you use AbortController?

124. Worker Thread solution.

Easy answer: Move CPU-heavy computation into a worker or worker pool and communicate input/output through messages.

10-year insight: Use pooling, bounded concurrency and failure handling instead of spawning unlimited workers.

Likely cross-question: How would you return worker errors?

Final 10-Year Interview Differentiators

- Never stop at a definition. Add one production implication.
- Say what breaks under load, not only what works in a demo.
- Use numbers carefully: latency, throughput, memory, CPU and concurrency.
- Explain trade-offs instead of claiming one approach is always best.
- For async systems, always think about timeout, retry, cancellation and backpressure.
- For databases, always think about connection pools, indexes, transactions and failure.
- For security, think in layers rather than one package.
- For scaling, think about dependencies and shared state, not only application replicas.
- For performance, measure before optimizing.
- For senior coding rounds, explain complexity, edge cases and production concerns after writing the solution.

Best answer pattern: “At a high level... Internally... In production I would... The trade-off is... If scale/failure increases, I would...”